

Transaction processing and Concurrency control

▼ Key Concepts

1. Transaction:

- A **logical unit** of database processing.
- It comprises one or more database operations (retrieval, insertion, deletion, update).
- It must be **completed in its entirety** to ensure correctness. This is the "all or nothing" principle.

2. Transaction Processing Systems (TPS):

- Systems characterized by:
 - Large databases.
 - Hundreds of concurrent users.
 - High availability and fast response time requirements.
- **Examples:** Airline reservations, banking, online retail, stock markets.

3. The Two Main Problems:

- **Concurrency Control:** The problem that arises when multiple transactions execute simultaneously and **interfere** with each other, potentially leading to incorrect data.
- **Recovery:** The problem of restoring the database to a correct state after various types of **failures** (e.g., system crashes, hardware faults).

▼ Introduction to Transaction Processing

Transaction processing ensures that **database operations** are executed reliably, even when multiple users are interacting with the database or when failures occur.

Two major issues are covered:

1. **Concurrency control** – managing multiple transactions executing at the same time.

2. **Recovery** – restoring the database after failures (like power loss or system crashes).
-

♦ 2. Single-User vs Multiuser Systems (Section 21.1.1)

- A **single-user DBMS** allows only one user at a time (e.g., a local personal database).
- A **multiuser DBMS** allows many users to access and modify data **concurrently**.

Examples of multiuser systems:

- Airline reservation systems
- Banking systems
- Supermarkets
- Stock exchanges

These systems handle **hundreds or thousands of users** simultaneously — all performing **transactions** like deposits, bookings, or updates.

♦ 3. How Multiple Users Can Work Simultaneously

The concept that makes this possible is **multiprogramming**.

- The **Operating System (OS)** can run several processes “at the same time” by quickly switching between them.
- A **CPU** can handle only one process at a time, but **interleaving** gives the illusion of parallel execution.
- When one process waits for an **I/O operation** (e.g., reading from disk), the CPU switches to another process.

This keeps the CPU **busy** and **efficient**, rather than idle.

♦ 4. Interleaving and Parallelism

- **Interleaving:**

When a single CPU alternates between processes (A and B), executing a few instructions from one before switching to another.

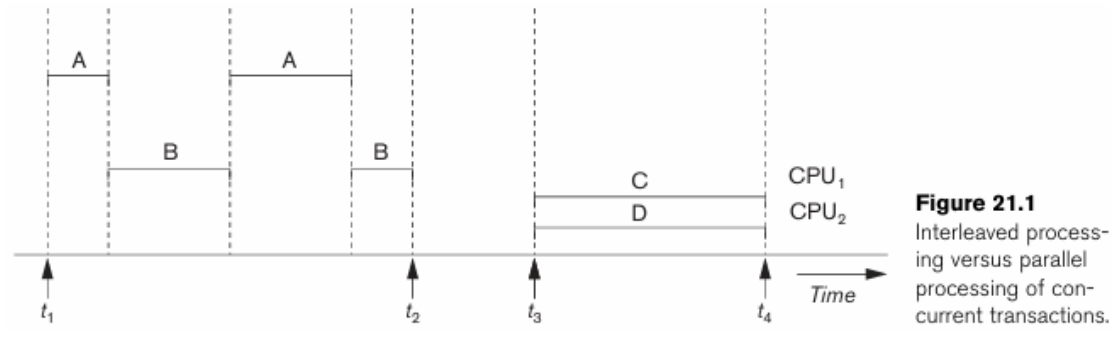
→ This is **concurrent execution** on a **single CPU**.

- **Parallel processing:**

When multiple CPUs (e.g., CPU1, CPU2) execute **different processes simultaneously** (like C and D).

→ True **parallel concurrency**.

Most database concurrency theories are based on the interleaved model, assuming one CPU but multiple transactions sharing it.



▼ Mechanics of a transaction

◆ 1. What Is a Transaction?

A **transaction** is:

An executing program that forms a logical unit of database processing.

It consists of one or more database operations such as:

- **Insertion**
- **Deletion**
- **Modification**
- **Retrieval**

* Example

If you transfer ₹1000 from Account A to Account B:

```
read(A)
A = A - 1000
write(A)
read(B)
B = B + 1000
```

```
write(B)
```

All of these together form **one transaction** — either **all succeed** or **none** should (atomicity).

◆ 2. Specifying Transaction Boundaries

- In an **application program**, a transaction is defined by:

```
BEGIN TRANSACTION
...
END TRANSACTION
```

- All database operations between these two statements are considered part of a **single transaction**.
- A single program can have **multiple transactions**.

◆ 3. Types of Transactions

Type	Description
Read-only transaction	Only retrieves data (no updates). Example: <code>SELECT * FROM employees</code>
Read-write transaction	Both reads and updates data. Example: bank transfer, order processing

◆ 4. Database Items and Granularity

A **database** is viewed as a **collection of named data items**, and each item's **size** is called its **granularity**.

Granularity	Example
Large	Whole disk block
Medium	Database record
Small	Attribute/field (e.g., employee name)

The concepts of transaction processing (reading/writing) are **independent of the granularity**.

Each data item has a **unique identifier** — such as a **disk block address** — but programmers usually access it via **field or table names**.

◆ 5. The Basic Operations

1 `read_item(X)`

Reads a database item **X** into a program variable (also named **X** for simplicity).

Steps:

1. Locate the disk block containing **X**.
 2. Copy that block into a **main memory buffer** (if not already there).
 3. Copy **X** from that buffer into the **program variable**.
-

2 `write_item(X)`

Writes the value of program variable **X** back into the database item **X**.

Steps:

1. Locate the disk block containing **X**.
2. Copy that block into a **main memory buffer** (if not already there).
3. Copy **X** from the program variable into the correct position in the buffer.
4. **Write the buffer back to disk** — either immediately or later.

|  Step 4 is the point where the actual database update happens.

Sometimes the DBMS delays writing to disk (to optimize performance or group updates).

The **recovery manager** and **operating system** together decide **when** to write modified buffers to disk.

◆ 6. DBMS Buffers and Replacement Policy

- The **DBMS** maintains a **database cache** (a set of memory buffers).
- Each buffer holds one **disk block** containing some database items.
- When buffers are full and a new block is needed:

- A **buffer replacement policy** (like Least Recently Used – LRU) chooses which one to evict.
- If that buffer has been **modified (dirty)**, it must first be **written back to disk**.

This ensures:

- Data integrity
- Efficient memory use
- Safe recovery in case of crashes

◆ 7. Read-set and Write-set

For any transaction **T**, we define:

Term	Meaning	Example
Read-set(T)	All data items that T reads	e.g., {X, Y}
Write-set(T)	All data items that T updates	e.g., {X, Y}

Example from Figure 21.2 (mentioned in your text):

- Transaction **T₁** reads and writes items **X** and **Y**
→ Read-set = {X, Y}, Write-set = {X, Y}

◆ 8. Why This Matters

When multiple transactions run **concurrently**, they may read/write **the same data items**.

If not controlled properly, this leads to:

- **Inconsistency**
- **Lost updates**
- **Dirty reads**, etc.

👉 These problems motivate **concurrency control mechanisms** (like locks, timestamps, serializability), discussed in later sections.

✅ In Summary

Concept	Meaning
Transaction	Logical unit of work with read/write operations
Granularity	Size of data item (field, record, or block)
read_item(X)	Fetch X from disk → memory → program
write_item(X)	Copy X from program → memory buffer → disk
Buffer	Temporary storage in main memory
Read-set / Write-set	Lists of all items read or written by the transaction
Buffer Replacement Policy	Determines which memory block to replace when full

▼ Need of Concurrency Control

◆ Why Concurrency Control Is Needed

When **multiple transactions** access and modify the **same data items** concurrently, **uncontrolled interleaving** (mixing of operations) can lead to **incorrect or inconsistent results**.

To illustrate this, the text uses a **simplified airline reservation system**:

- Each **flight** is stored as a **record** in the database.
- Each record has a field for **number of reserved seats**, represented by data items like **X** and **Y**.

✚ Example Transactions

✚ Transaction T₁ — Transfer N seats from Flight X → Flight Y

```
read_item(X);
X := X - N;
write_item(X);
read_item(Y);
Y := Y + N;
write_item(Y);
```

✚ Transaction T₂ — Reserve M seats on Flight X

```
read_item(X);
X := X + M;
```

```
write_item(X);
```

These transactions are simple — but when they **run concurrently**, serious problems can occur.

♦ 1. The Lost Update Problem

Definition:

Occurs when **two transactions update the same data item**, but one of the updates **gets overwritten (lost)** due to improper interleaving.

Example (T₁ and T₂):

Let's say:

- Initially: $X = 80$
- T₁ transfers $N = 5$ seats (so X should become 75)
- T₂ reserves $M = 4$ seats (so X should become 79)
- Correct final value: X = 79**

Interleaving:

```
T2: read_item(X) → X = 80
T1: read_item(X) → X = 80
T1: X := X - 5 → X = 75
T1: write_item(X) → X = 75
T2: X := X + 4 → X = 84
T2: write_item(X) → X = 84
```

Result:

The update made by T₁ (**X = 75**) is **lost**, and the final value becomes **X = 84** (wrong).

Why It Happens:

T₂ read the old value of X **before** T₁'s update was written, and then overwrote it.

♦ 2. The Temporary Update (or Dirty Read) Problem

Definition:

Occurs when a transaction **reads data that was written by another transaction that later fails** (and rolls back).

The data read is **"dirty"** — it's not valid anymore.

Example:

```
T1: read_item(X)
T1: X := X - 5
T1: write_item(X)
T1: (fails)
T2: read_item(X)
```

If **T1 fails** after updating X but before committing, the system must restore X to its **original value**.

But **T2 has already read the temporary, incorrect value**.

Result:

T2 uses a **dirty (uncommitted)** value → causes **inconsistency**.

◆ 3. The Incorrect Summary Problem

Definition:

Occurs when one transaction is computing an **aggregate summary (like sum, average, or count)** while other transactions are **updating** the data being summarized.

Example:

- T₁ transfers **N** seats from **X** to **Y**
- T₃ computes the total seats across all flights

Interleaving:

```
T1: read_item(X)
T1: X := X - N
T1: write_item(X)

T3: read_item(X)
T3: sum := sum + X
```

```

T3: read_item(Y)
T3: sum := sum + Y

T1: read_item(Y)
T1: Y := Y + N
T1: write_item(Y)

```

💥 Result:

- T₃ reads X **after** seats were subtracted (smaller)
- T₃ reads Y **before** seats were added (larger)
- So the total ($X + Y$) is **off by N**

♦ 4. The Unrepeatable Read Problem

🧠 Definition:

Occurs when a transaction **reads the same data item twice** and **gets different values** because another transaction modified it in between.

⚙️ Example:

- A customer checks **seat availability** twice during a booking transaction.
- Another transaction books seats between the two reads.

✗ Interleaving:

```

T4: read_item(X)   → X = 80
T2: read_item(X)
T2: X := X + 4
T2: write_item(X)
T4: read_item(X)   → X = 84

```

💥 Result:

The same transaction (T₄) gets **two different results** for the same query.

→ Leads to **inconsistent or confusing behavior**.

⚖️ Summary Table

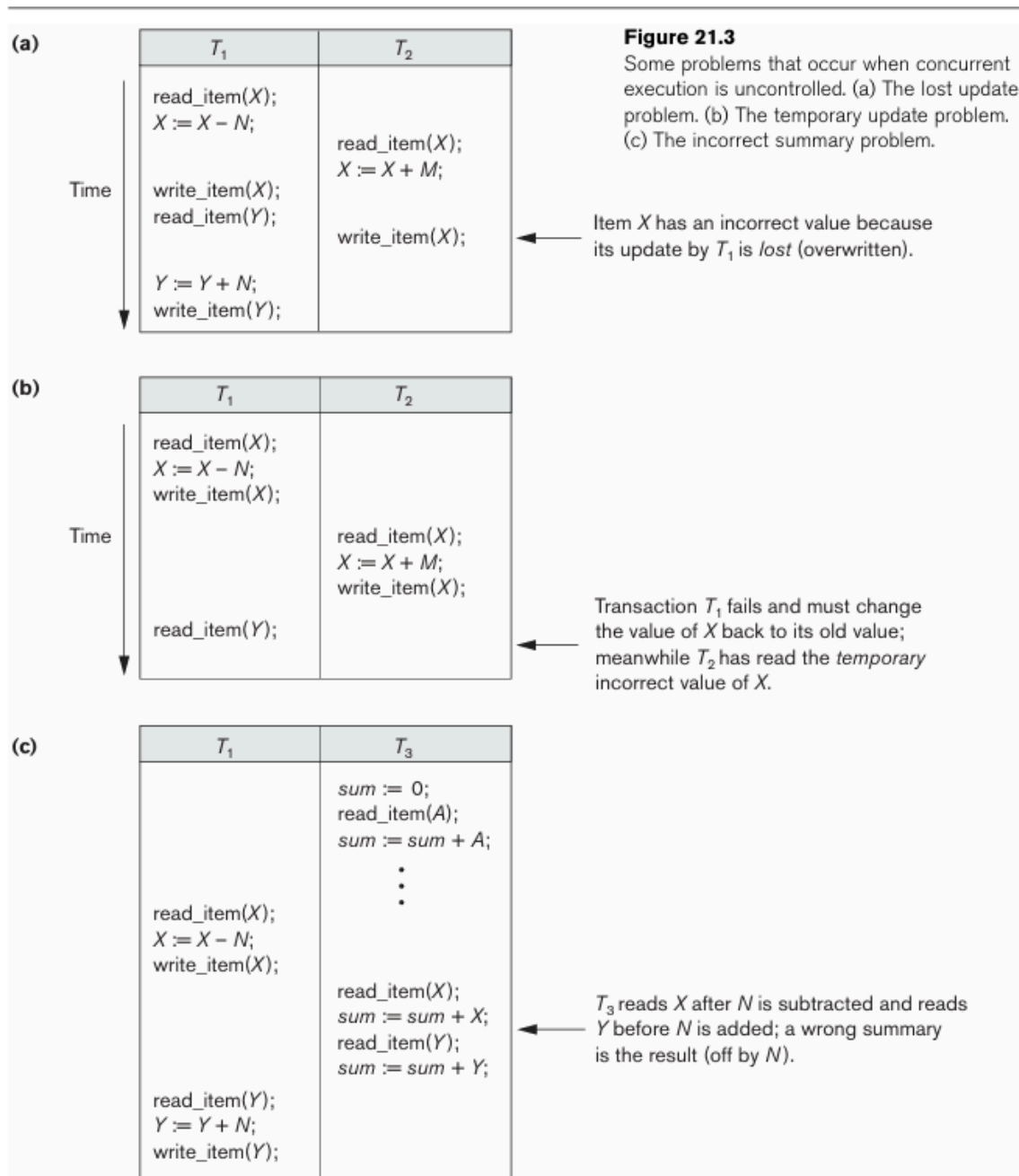
Problem	Description	Example Scenario	Consequence
Lost Update	One update overwrites another	T ₁ and T ₂ both update X	Final value incorrect
Temporary Update / Dirty Read	One transaction reads data written by a failed one	T ₂ reads X updated by failed T ₁	T ₂ uses invalid data
Incorrect Summary	Aggregate computed during concurrent updates	T ₃ sums while T ₁ transfers	Total is inaccurate
Unrepeatable Read	Same data read twice gives different values	Seat check before and after another booking	Inconsistent view

Why Concurrency Control Is Needed

Without control, concurrent transactions can:

- Violate **database consistency**
- Lose updates or read incorrect data
- Produce **unreliable or contradictory results**

✓ Therefore, **Concurrency Control** techniques (like **locking**, **timestamp ordering**, **serializability**, etc.) are essential to ensure the database behaves **as if transactions were executed one at a time (serially)** — even though they actually run concurrently



▼ Why Recovery Is Needed

Whenever you run a **transaction** in a database, the DBMS has one crucial responsibility:

👉 Either all operations of that transaction are completed successfully and saved permanently (commit)
or **none** of them take effect (**abort / rollback**).

This is part of the **ACID properties** — specifically **Atomicity** and **Durability**.

⚙️ Example

Let's say you have a banking database:

- Transaction T1: Transfer ₹500 from **A** to **B**

```
read(A)
A := A - 500
write(A)
read(B)
B := B + 500
write(B)
```

Now suppose after subtracting ₹500 from A, the system **crashes** before adding it to B.

If no recovery is done, you'll lose ₹500 — ❌ the database becomes inconsistent.

That's why recovery is needed — to **undo partial changes** (rollback) or **redo committed ones** after failure.

⚠️ Types of Failures

DBMS recovery mechanisms exist to handle different **types of failures** that can occur during transaction execution.

Let's go through them one by one 👉

1 System Crash (Computer Failure)

- Example: Power outage, OS crash, hardware malfunction, etc.
- Effect: All transactions in **main memory** are lost; data in **disk** remains.
- DBMS must use its **log file** to restore the database to a consistent state.

2 Transaction or System Error

- Example: Division by zero, invalid input, or logical bug.
 - Effect: Transaction stops abnormally in the middle.
 - DBMS must **abort** the transaction and **undo** any changes it made.
-

3 Local Errors or Exception Conditions

- Example: "Insufficient funds" or "record not found."
 - These are **expected exceptions** that cause the transaction to **cancel itself** safely.
 - They are **not considered failures** if handled correctly (because the program knows how to deal with them).
-

4 Concurrency Control Enforcement

- Example: Deadlock or serializability violation.
 - DBMS might **force a transaction to abort** to keep the database consistent.
 - The aborted transaction is usually **restarted automatically** later.
-

5 Disk Failure

- Example: Disk block corruption, read/write head crash.
 - Effect: Some data on the disk is lost or unreadable.
 - Recovery is much harder — may require **backup restoration**.
-

6 Physical Problems / Catastrophes

- Example: Fire, theft, power failure, sabotage, wrong tape mounting, etc.
 - These are **rare but severe**.
 - Recovery requires restoring data from **full backups** and **archived logs**.
-

Frequency

Type	Frequency	Typical Recovery Action
1–4	Common	Undo/redo using logs
5–6	Rare	Restore from backups

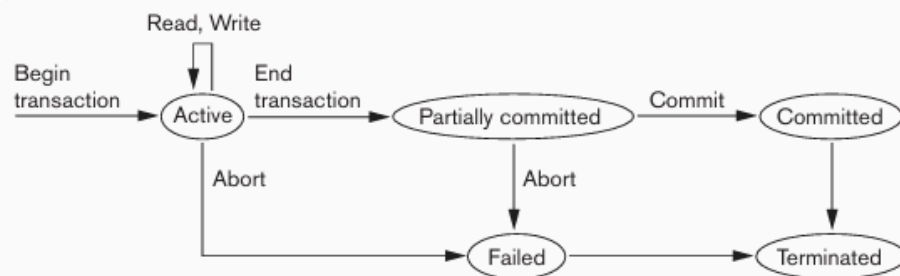
Summary

Concept	Meaning
Commit	All transaction operations complete successfully; effects are made permanent.
Abort	Transaction fails; all its partial effects are undone.
Recovery	Process of restoring the database to a consistent state after a failure.
Why needed?	To ensure Atomicity (all or nothing) and Durability (once done, stays done).

▼ Lifecycle of a transaction

Figure 21.4

State transition diagram illustrating the states for transaction execution.



⁵Optimistic concurrency control (see Section 22.4) also requires that certain checks are made at this point to ensure that the transaction did not interfere with other executing transactions.

When a transaction runs, the DBMS must **keep track of what stage it's in** — whether it's just started, doing work, finished successfully, or failed midway. This helps the **recovery manager** decide what to do if something goes wrong (e.g., crash, error, abort).

Basic Operations Used in Transaction Processing

The DBMS defines several operations to manage transactions:

Operation	Meaning
BEGIN_TRANSACTION	Marks the start of a new transaction.

Operation	Meaning
READ(X)	Read item X from the database into memory.
WRITE(X)	Write the modified value of X back to the database.
END_TRANSACTION	Signals that the transaction has finished all its read/write operations. Now DBMS must decide whether to commit or abort .
COMMIT_TRANSACTION	Means everything succeeded → all updates can be permanently saved to the database.
ROLLBACK / ABORT	Means something went wrong → all updates made so far must be undone (rolled back) .

Transaction States (Life Cycle)

The diagram (Figure 21.4) in the book shows **how a transaction moves between states** depending on what happens during execution.

Let's explain each one clearly 📌

1. Active State

- The transaction is **executing** its statements (e.g., reading, writing, calculations).
- It's not done yet.
- Example:

```
BEGIN_TRANSACTION
READ(A)
A := A - 500
WRITE(A)
```

2. Partially Committed State

- The transaction has finished all its steps successfully and executed `END_TRANSACTION`.
- The DBMS now performs **final checks** before making the changes permanent.
- It checks things like:
 - Are logs written correctly?
 - Are there concurrency conflicts (serializability, deadlocks)?

- Is the system stable enough to record changes?

If everything is OK → move to **Committed**.

If not → move to **Failed**.

3. Committed State

- The transaction has **reached its commit point**.
 - All changes are now **permanently recorded** in the database (even if a crash occurs immediately after).
 - The system can safely remove the transaction from active memory.
-

4. Failed State

- Something went wrong during execution:
 - Logical error (e.g., divide by zero),
 - System crash,
 - Concurrency issue (e.g., deadlock),
 - Or manual abort.
 - The transaction **cannot continue**.
-

5. Aborted / Rolled Back State

- DBMS **undoes all changes** the transaction made (using the system log).
 - Once undone, the transaction either:
 - Is **terminated**, or
 - Is **restarted** as a new transaction.
-

6. Terminated State

- The transaction's life is completely over.
 - All temporary data, locks, and transaction records are removed from memory.
 - Nothing more will happen with this instance.
-

State Transition Summary

From State	Operation	Next State	Meaning
—	BEGIN_TRANSACTION	Active	Transaction starts
Active	Normal execution	Active	Still running
Active	END_TRANSACTION	Partially Committed	Finished all statements
Partially Committed	COMMIT_TRANSACTION	Committed → Terminated	Success ✓
Active / Partially Committed	ROLLBACK / ABORT	Failed → Terminated	Failure ✗

🧠 Why These States Are Important

1. Help the **recovery manager** decide what to redo or undo after a crash.
2. Allow the **concurrency control** system to coordinate multiple transactions safely.
3. Ensure **atomicity** → either the whole transaction happens or none of it does.

▼ System Log

◆ What Is the System Log?

The **system log** is a **special file** maintained by the DBMS to record **everything that happens inside transactions** —

especially any action that **changes the database**.

🧠 Think of it as the **“black box recorder”** of the database — just like an airplane's flight recorder.

If the system crashes, the DBMS can look into the log and **rebuild** what was happening right before the failure.

⚙️ Key Characteristics

- The log is a **sequential, append-only file** → new records are **only added at the end**.
- It is stored on **disk** (non-volatile memory) → so it **survives system crashes**.
- Usually, the **latest part of the log** is kept in a **main memory buffer** for performance reasons.

- When the buffer fills up, it's written (flushed) to disk.
- Periodically, the log file on disk is **backed up** to external storage (like tape or cloud backup).
 - This protects it from **catastrophic failures** (like disk damage or fire).

Types of Log Records

Each time something important happens in a transaction, the DBMS adds a **log record (entry)** to the log.

Each entry stores a **transaction ID (T)** and other details.

Here's what each entry means 📌

Log Record	Description
<code>[start_transaction, T]</code>	Marks that transaction T has started .
<code>[write_item, T, X, old_value, new_value]</code>	Transaction T updated item X . The log records both: • <code>old_value</code> → used for undo during rollback. • <code>new_value</code> → used for redo if system crashed before commit.
<code>[read_item, T, X]</code>	Transaction T read item X (usually optional, not needed for recovery).
<code>[commit, T]</code>	Transaction T finished successfully → its changes are now permanent .
<code>[abort, T]</code>	Transaction T failed or was rolled back → changes must be undone .

Why the Log Is Essential for Recovery

Suppose the system crashes halfway through some transactions.

When the DBMS restarts, it uses the **log** to restore consistency:

Undo (Rollback)

If a transaction **did not commit** before the crash →

→ Undo all its effects (using `old_value` from `[write_item]` entries).

This ensures that **incomplete work is erased**.

Redo

If a transaction **committed** but changes weren't yet written to disk →

→ Redo all its updates (using `new_value`).

This ensures that **committed work isn't lost**.

So the log provides the data needed to perform **both undo and redo** operations.

Example

Let's look at a small example.

Suppose:

```
Transaction T1:
  read_item(X)
  X := X + 100
  write_item(X)
```

The **log entries** would look like:

```
[start_transaction, T1]
[write_item, T1, X, 500, 600]    -- X changed from 500 → 600
[commit, T1]
```

Now imagine the system crashes:

-  If `[commit, T1]` is in the log → DBMS will **redo** the change (to ensure $X = 600$).
-  If `[commit, T1]` is not found → DBMS will **undo** the change (restore $X = 500$).

What About READ Operations?

Most recovery systems **don't record READ operations** in the log because:

- Reads don't change data.
- They're not needed to restore consistency.

However, in **auditing systems** (e.g., for security or analysis), READ logs **might be included**.

Simplified Diagram of Logging & Recovery

Main Memory (RAM)

↓

Log Buffer → periodically written → Disk Log File → periodically backed up → Archival Storage

During crash:

- Read log file
- Undo uncommitted transactions
- Redo committed ones

✓ Summary

Concept	Description
Purpose	To record all transaction actions for recovery
Stored Where	On disk (append-only)
Key Entries	start, write (with old & new values), commit, abort
Used For	Undo and Redo during recovery
Protection Against	System crashes, transaction failures
Not Protected Against	Catastrophic disk failure (hence backup needed)

▼ Commit Point

1 Transaction execution

A transaction (say **T**) performs operations like:

```
read_item(X)
X := X - 10
write_item(X)
```

During this time:

- The transaction is making temporary changes in memory (buffers, cache).
- These changes are **not yet permanent** in the database on disk.

2 Logging every action

Before any actual data is written to disk, the DBMS keeps a **log file** (a special file used for recovery).

Each log entry looks like this:

```
[start_transaction, T]
[write_item, T, X, old_value, new_value]
[commit, T]
```

This log file helps the system **recover** if a crash happens.

For example:

- If `commit` record exists → transaction's work can be **redone**.
- If only `start_transaction` exists → transaction must be **undone**.

3 The Commit Point

A transaction **reaches its commit point** when:

- ✓ All its database operations have executed successfully, and
- ✓ All its effects (WRITE operations) are **recorded in the log**, safely on disk.

Once this happens:

```
[commit, T]
```

is written to the log.

From now on, `T` is said to be **committed** — its changes must never be lost.

4 Force-writing the log buffer

Logs are often temporarily kept in **main memory (RAM)** for efficiency — writing to disk after every entry is too slow.

But there's a risk:

If the system crashes before the log buffer is saved to disk, the log entries in RAM are lost.

So right **before commit**, the system **force-writes** the log buffer to disk — this ensures the recovery system can see:

```
[start_transaction, T]
[write_item, T, ...]
[commit, T]
```

✓ This guarantees the transaction's actions can be **redone** after a crash.

⚠ Without force-writing, the system could lose evidence that **T** ever committed.

⚙ Example in Practice

Step	Action	Description
1	T1 starts	<code>[start_transaction, T1]</code> in log
2	T1 updates X	<code>[write_item, T1, X, 10, 20]</code> in log
3	T1 ready to commit	Force-write all log records (including above) to disk
4	T1 commits	<code>[commit, T1]</code> written and force-written to disk
5	System crash	Recovery looks at log: since <code>[commit, T1]</code> exists → redo T1

💬 In short:

- **Commit point** = when the transaction's all actions are done and safely logged to disk.
- **Force-writing the log buffer** ensures no log info is lost before commit.
- After commit → transaction's changes are *permanent*.
- Before commit → transaction might be rolled back if failure happens.

▼ ACID Properties of Transactions

Property	Meaning	How It's Enforced
A – Atomicity	A transaction is all or nothing . Either it completes fully or has no effect at all.	The recovery system ensures that if a transaction fails mid-way, any partial changes are rolled back .

Property	Meaning	How It's Enforced
C – Consistency	A transaction should preserve database consistency . After it finishes, the database must remain in a valid state according to all constraints (primary keys, foreign keys, domain rules, etc.).	Usually the programmer ensures consistency, or the DBMS enforces integrity constraints .
I – Isolation	A transaction should appear to execute alone , even if multiple transactions run concurrently. Its intermediate results should not affect other transactions .	The concurrency control subsystem enforces isolation. Levels of isolation: • Level 0 – prevents overwriting dirty reads. • Level 1 – prevents lost updates. • Level 2 – no lost updates, no dirty reads. • Level 3 (true isolation) – adds repeatable reads.
D – Durability	Once a transaction commits, its changes must persist , even if the system crashes immediately afterward.	The recovery subsystem ensures changes are written to disk and can be redone after a crash.

♦ Examples

1. Atomicity:

- Transferring \$100 from Account A to Account B.
- Either both debit and credit occur, or neither happens.

2. Consistency:

- If the total balance in all accounts must be \$10,000, any transaction should not break this rule.

3. Isolation:

- If two people withdraw from the same account concurrently, each should see a **consistent balance**.

4. Durability:

- Once the transaction updates the account balances and commits, the changes must survive a **system crash**.

Recall that a database state is a collection of all the stored data items (values) in the database at a given point in time. A consistent state of the database satisfies

the constraints specified in the schema as well as any other constraints on the database that should hold.

▼ Characterizing Schedules Based on Recoverability

▼ Intro

What is a Schedule?

- When **multiple transactions execute concurrently**, their operations (reads and writes) are **interleaved**.
- The **sequence of all operations from all transactions**, in the order they actually execute, is called a **schedule** (or history).

| Think of it like a timeline of all actions performed by all transactions.

Example:

Suppose we have two transactions:

```
T1: read(X), write(X)
T2: read(Y), write(Y)
```

A possible **interleaved schedule** could be:

```
T1: read(X)
T2: read(Y)
T1: write(X)
T2: write(Y)
```

◆ Why Schedules Matter for Recovery

- When a **system failure** occurs, we need to know which operations have executed and in what order.
- Certain types of schedules are **better for recovery**, meaning we can undo or redo transactions safely without leaving the database in an inconsistent state.
- These are called **recoverable schedules**.

Key idea:

- The **DBMS uses the log and the schedule** to determine which transactions to commit, undo, or redo during recovery.

▼ Schedules (Histories) of Transactions

A **schedule** (or **history**) is basically a **timeline of operations from multiple transactions** executed concurrently.

- Suppose you have transactions `T1, T2, ..., Tn`.
- A **schedule S** is an **ordering of all operations** from these transactions.
- Operations from **different transactions can be interleaved**.
- **Important:** Operations **within the same transaction** must appear **in the order they were written** in the transaction.

◆ Total Ordering vs Partial Ordering

- **Total ordering:** Every pair of operations in the schedule has a defined order.
- **Partial ordering:** Only conflicting operations or operations from the same transaction have a defined order; nonconflicting operations can appear in any order.

| For now, we assume total ordering to simplify reasoning.

◆ Shorthand Notation for Schedules

- Operations can be represented as:
 - `b` = `begin_transaction`
 - `r` = `read_item`
 - `w` = `write_item`
 - `e` = `end_transaction`
 - `c` = `commit`
 - `a` = `abort`
- Each operation is **subscripted** with the transaction id, and data items are in parentheses.

Example:

For transactions T1 and T2:

```
Sa: r1(X); r2(X); w1(X); r1(Y); w2(X); w1(Y)
```

This shows the **interleaved operations** in schedule **Sa**.

◆ Conflicting Operations

Two operations **conflict** if all three conditions are met:

1. They belong to **different transactions**.
2. They access the **same database item X**.
3. At least **one is a write operation**.

Examples:

Operations	Conflict?	Reason
r1(X), r2(X)	✗ No	Both are reads (no conflict)
r1(X), w2(X)	✓ Yes	One is write, same item
w1(X), w2(X)	✓ Yes	Both write, same item (write-write conflict)
w1(X), w2(Y)	✗ No	Different items
r1(X), w1(X)	✗ No	Same transaction

Intuition: Conflicting operations are ones where changing the order changes the final outcome.

◆ Complete Schedule

A schedule S is **complete** if:

1. It contains **all operations** of all transactions, including the **last commit or abort** for each transaction.
2. Operations of a transaction appear **in the same order** as in the original transaction.
3. **Conflicting operations** have a defined order in the schedule.

Non-conflicting operations can appear in any order.

◆ Committed Projection C(S)

- In real systems, transactions are **constantly submitted**, so schedules are often incomplete.
- The **committed projection C(S)** of a schedule S includes **only the operations of transactions that have committed**.
- This is useful for **recovery and consistency checking**, because **aborted transactions do not affect the database permanently**.

✓ Key Takeaways

- A **schedule** = interleaved timeline of operations.
- **Conflicts** = operations from different transactions on the same data item, with at least one write.
- **Complete schedule** = contains all operations + commits/aborts + defined order for conflicting ops.
- **Committed projection** = only the committed operations of transactions, important for recovery.

▼ Committed Projection C(S) Explained

When a schedule SSS contains **many interleaved transactions**, not all transactions may **successfully complete** (commit). Some may **abort or fail**.

- **Committed transactions**: Transactions that successfully finished and whose changes **must be permanently applied** to the database.
- **Aborted or failed transactions**: Transactions that did not finish successfully and whose changes **should not affect the database**.

The **committed projection C(S)** is:

A version of the schedule SSS that keeps only the operations from committed transactions and ignores the operations of aborted/failed transactions.

◆ Why is it needed?

- Recovery only cares about **what actually happened permanently** in the database.
- If a transaction failed or was aborted, its operations should **not be considered**, because they are **rolled back**.
- C(S)C(S)C(S) is a simplified schedule showing only the **committed transactions**, making it easier to reason about consistency and recoverability.

♦ Example

Suppose we have schedule SSS:

```
S: b1; r1(X); w1(X); b2; r2(X); w2(X); c1; r3(Y); w3(Y); a2
```

- T1 committed (c1)
- T2 aborted (a2)
- T3 is still running

Then the **committed projection** C(S)C(S)C(S) is:

```
C(S): b1; r1(X); w1(X); c1
```

Only T1's operations appear because it committed. T2's aborted operations are ignored, and T3 hasn't committed yet, so it's also ignored.

💡 Think of it this way:

C(S)C(S)C(S) = "the part of the schedule that actually counts in the database after all failures and rollbacks."

▼ Characterizing Schedules Based on Recoverability

This section is about **classifying schedules based on recoverability and how safe**

they are for recovery.

◆ 1. Recoverable Schedules

Definition:

A schedule S is **recoverable** if no transaction commits until all transactions whose values it has read have committed.

Why?

- Suppose T_2 reads data written by T_1 .
- If T_2 commits **before** T_1 commits and T_1 later aborts, then the value that T_2 used is invalid.
- T_2 would then have to be undone **after it has already committed**, which violates durability.

Example:

- Schedule S_c :

SCSS

Copy code

```
r1(X); w1(X); r2(X); r1(Y); w2(X); c2; a1
```

- T_2 reads X from T_1 but commits **before** T_1 commits.
- If T_1 aborts ($a1$), T_2 has committed using a now-invalid value → **nonrecoverable**.
- To fix it: Postpone T_2 's commit until after T_1 commits → S_d or abort T_2 before it commits → S_e .

✓ So in a recoverable schedule, committed transactions never need to be rolled back.

◆ 2. Cascadeless Schedules (Avoiding Cascading Rollback)

Definition:

A schedule is **cascadeless** if every transaction reads only items written by committed transactions.

Why?

- In a recoverable schedule, **uncommitted transactions may still be read**.
- If a transaction that is read later aborts, all dependent transactions also need to abort → cascading rollback.

Example:

- Schedule S_e :

SCSS

Copy code

```
r1(X); w1(X); r2(X); ...; a1
```

- T_2 reads X from T_1 (which aborts).
- T_2 must also roll back → **cascading rollback**.
- To make it **cascadeless**, delay $r2(X)$ until after T_1 commits or aborts.

✓ Cascadeless schedules **avoid cascading aborts**, but may delay transactions.

◆ 3. Strict Schedules

Definition:

A schedule is **strict** if no transaction can read or write an item **X** until the last transaction that wrote **X** has committed or aborted.

Why stricter than cascadeless?

- In strict schedules, recovery is simple:
 - If a transaction aborts, just restore the **old value (before image)** of its writes.
- No other transaction can interfere with this value before it's committed.

Example:

- Schedule **Sf** :

scss

 Copy code

```
w1(X,5); w2(X,8); a1
```

- **T2** wrote **X** before **T1** committed or aborted → strictness violated.
- Simple undo of **T1**'s write would incorrectly reset **X** to old value, ignoring **T2**'s write.

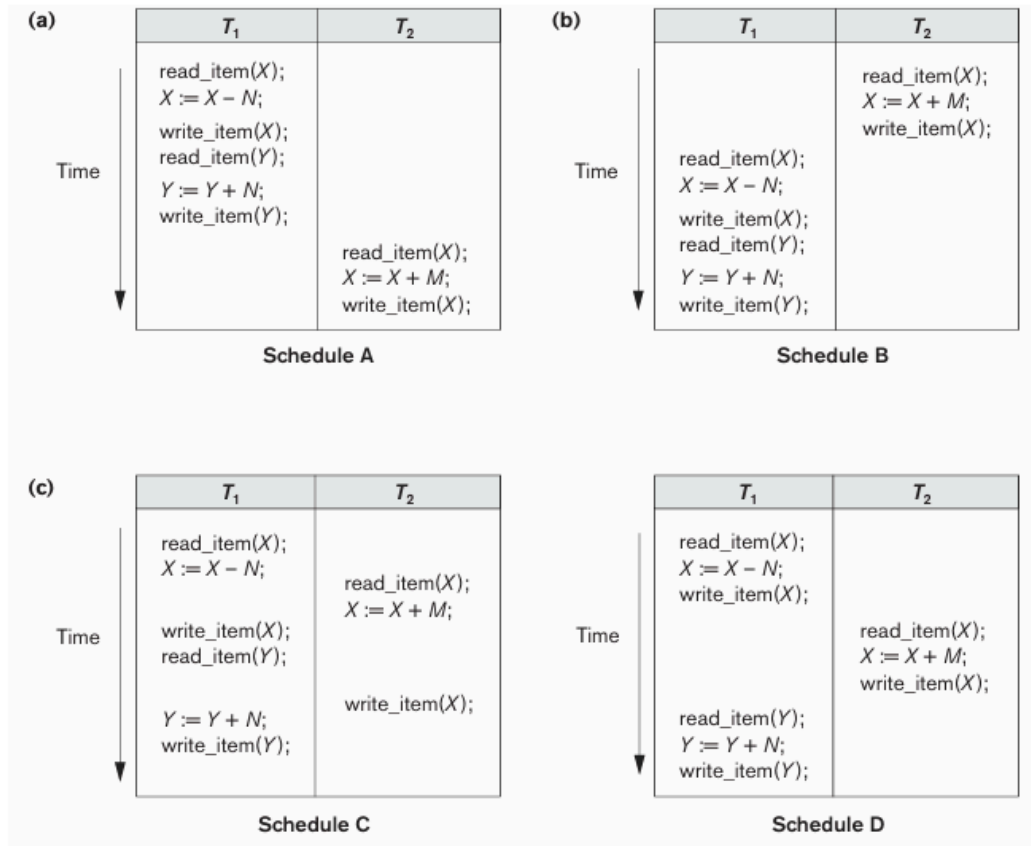
✓ Every **strict** schedule is **cascadeless**, and every **cascadeless** schedule is **recoverable**.

Schedule Type	Properties		
Recoverable	No committed transaction needs rollback. May have cascading aborts.		
Cascadeless	Subset of recoverable. Avoids cascading rollbacks.		
Strict	Subset of cascadeless. No reads/writes until previous transaction commits. Simple recovery.		

▼ Serializability

Figure 21.5

Examples of serial and nonserial schedules involving transactions T_1 and T_2 . (a) Serial schedule A: T_1 followed by T_2 . (b) Serial schedule B: T_2 followed by T_1 . (c) Two nonserial schedules C and D with interleaving of operations.



1. Serial Schedules

Definition:

- A **serial schedule** executes transactions **one after another**, without interleaving their operations.
- Example with transactions T_1 and T_2 :
 1. **Schedule A (Serial)**: Execute all of T_1 first, then all of T_2 .
 2. **Schedule B (Serial)**: Execute all of T_2 first, then all of T_1 .

Key point: Serial schedules are **always correct**, because each transaction sees the database as if it were alone.

♦ 2. Nonserial Schedules (Interleaving)

- In practice, the DBMS may interleave operations from multiple transactions to improve performance.
- Example: **T1** and **T2** operations mixed in some order → Schedules C and D in the figure.

Problem: Interleaving can lead to **incorrect results** if not controlled.

♦ 3. Serializability

Definition:

| A schedule is serializable if its result is equivalent to some serial schedule.

- Even if operations are interleaved, if the final state of the database is **the same as one of the possible serial schedules**, the interleaved schedule is **correct**.
- Serializability ensures **correctness** of concurrent execution without forcing transactions to run one after the other.

Why important:

- Allows **concurrency** (better performance) without violating database consistency.
 - Ensures **results are predictable**, like in serial execution.
-

💡 **Intuition:**

- Think of a bank: Two transactions, **T1** deposits and **T2** withdraws.
- Interleaving reads/writes could change the final balance.
- If the interleaved schedule produces the same final balance as either "T1 then T2" or "T2 then T1", it's **serializable** and safe.

▼ Serial, Nonserial, and Conflict-Serializable Schedules

♦ 1. Serial vs Nonserial Schedules

➤ Serial Schedule

A **serial schedule** executes one transaction completely **before** another starts.

✅ No interleaving.

✅ Always correct (since each transaction runs in isolation).

Example:

```
Schedule A: T1 → T2  
r1(X), w1(X), r1(Y), w1(Y), c1  
r2(X), w2(X), c2
```

► Nonserial Schedule

A **nonserial schedule** interleaves operations of multiple transactions to improve performance.

⚠️ But interleaving may cause **inconsistency** if not controlled.

Example:

```
Schedule C:  
r1(X), r2(X), w1(X), w2(X), ...
```

♦ 2. Why Not Use Only Serial Schedules?

Because:

- They **waste CPU time** (if a transaction waits for I/O, CPU sits idle).
- They **reduce concurrency** — long transactions block others.

Hence, DBMS allows **interleaving** — but must ensure **correctness**.

♦ 3. Serializable Schedule

A schedule is serializable if its final effect on the database is the same as some serial schedule.

That means:

- Interleaving is allowed,
- But the end result must be as if transactions had run one by one.

👉 So, **serializable = logically equivalent to a serial schedule**.

◆ 4. Types of Equivalence

To say two schedules are “equivalent”, we need a precise definition of what that means.

There are two main definitions:

(a) Result Equivalence

- Two schedules are **result equivalent** if they produce the same final values in the database.

✗ Problem: This can happen **by coincidence**, so it’s not reliable.

(b) Conflict Equivalence (used in practice) ✓

Two schedules are **conflict equivalent** if:

- They have the **same operations**.
- The **order of conflicting operations** is the same in both schedules.

◆ 5. When Do Two Operations Conflict?

Two operations **conflict** if:

- They belong to **different transactions**.
- They access the **same data item**.
- At least **one is a write**.

Operation Pair	Conflicting?	Reason
r1(X), r2(X)	✗ No	Both read
r1(X), w2(X)	✓ Yes	Read–Write conflict
w1(X), r2(X)	✓ Yes	Write–Read conflict
w1(X), w2(X)	✓ Yes	Write–Write conflict

◆ 6. Conflict-Serializable Schedule

| A schedule is conflict-serializable if it is conflict equivalent to a serial schedule.

That means you can **swap nonconflicting operations** step by step until the schedule becomes serial.

♦ Example

Schedule D (from Figure 21.5):

```
T1: r1(X), w1(X), r1(Y), w1(Y)
T2: r2(X), w2(X)
```

✓ This schedule is **conflict-serializable**, equivalent to **T1 → T2**,

because:

- T1's writes happen before T2's reads/writes on same items.
- You can reorder nonconflicting ops (like Y operations) safely.

Schedule C:

```
r1(X); r2(X); w1(X); w2(X); ...
```

✗ Not serializable because:

- r2(X) conflicts with w1(X)
- w1(X) conflicts with w2(X)

and no rearrangement can make it equivalent to T1 → T2 or T2 → T1.

♦ 7. Summary Table

Type	Meaning	Allows Interleaving?	Always Correct?
Serial	One transaction at a time	✗	✓
Nonserial	Operations interleaved	✓	✗ (not always)
Conflict-Serializable	Interleaved but equivalent to serial	✓	✓

▼ Explanation

🧩 Step 1. The given schedule D

We have two transactions:

T1

```
r1(X), w1(X), r1(Y), w1(Y)
```

T2

```
r2(X), w2(X)
```

Step 2. Meaning of conflict

Two operations **conflict** if:

1. They belong to **different transactions** (T1 and T2),
2. They access the **same data item**, and
3. **At least one** of them is a **write**.

That means the following pairs **conflict**:

- **r1(X)** and **w2(X)** (read–write conflict on X)
- **w1(X)** and **r2(X)** (write–read conflict on X)
- **w1(X)** and **w2(X)** (write–write conflict on X)

Operations on **different items (like Y)** do **not** conflict.

Step 3. Look at the actual order in the schedule

Let's look at the sequence in **schedule D** (from your text):

```
T1: r1(X), w1(X), r1(Y), w1(Y)
T2: r2(X), w2(X)
```

→ In time order, that means:

```
r1(X)
w1(X)
r2(X)
w2(X)
```

r1(Y)
w1(Y)

🧩 Step 4. See who comes first in the conflicts

Now let's check the conflicting operations on **X**:

Data Item	T1 Operation	T2 Operation	Which Happens First?	Dependency
X	w1(X)	r2(X)	w1(X) before r2(X)	T1 → T2
X	w1(X)	w2(X)	w1(X) before w2(X)	T1 → T2

✅ All conflicts show that **T1 must come before T2** for this schedule to be equivalent to a serial one.

No conflicts suggest that T2 should come before T1.

So, the **serialization order** is **T1 → T2**.

🧩 Step 5. Why can we reorder safely?

- Operations that do **not conflict** can be freely swapped.
- The operations on **Y** (r1(Y), w1(Y)) are from T1 only — no T2 operations touch Y.
- Therefore, moving Y-related operations earlier or later doesn't change the outcome.

Hence, we can imagine reordering the schedule so that **all of T1's operations happen before T2's**, without changing any result.

That's why the schedule is **conflict-serializable**, and it's **equivalent to the serial schedule (T1 followed by T2)**.

✅ Final understanding

Type	Explanation
Conflict-serializable	Because all conflicts imply the same order (T1 before T2).
Equivalent serial order	T1 → T2

Type	Explanation
Non-conflicting ops (on Y)	Can be safely reordered — they don't affect the serialization.

▼ Testing for Conflict Serializability of a Schedule

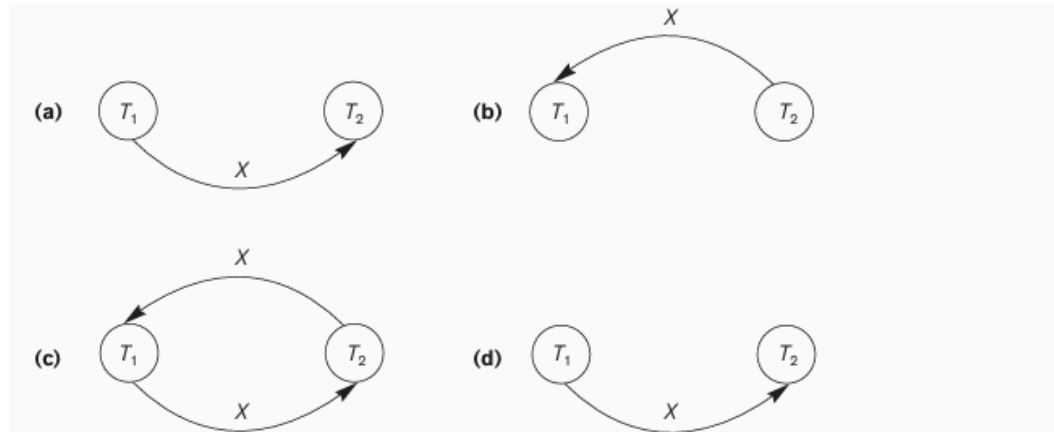


Figure 21.7

Constructing the precedence graphs for schedules A to D from Figure 21.5 to test for conflict serializability. (a) Precedence graph for serial schedule A. (b) Precedence graph for serial schedule B. (c) Precedence graph for schedule C (not serializable). (d) Precedence graph for schedule D (serializable, equivalent to schedule A).

Algorithm 21.1. Testing Conflict Serializability of a Schedule S

1. For each transaction T_i participating in schedule S , create a node labeled T_i in the precedence graph.
2. For each case in S where T_j executes a `read_item(X)` after T_i executes a `write_item(X)`, create an edge $(T_i \rightarrow T_j)$ in the precedence graph.
3. For each case in S where T_j executes a `write_item(X)` after T_i executes a `read_item(X)`, create an edge $(T_i \rightarrow T_j)$ in the precedence graph.
4. For each case in S where T_j executes a `write_item(X)` after T_i executes a `write_item(X)`, create an edge $(T_i \rightarrow T_j)$ in the precedence graph.
5. The schedule S is serializable if and only if the precedence graph has no cycles.

What is being tested?

We have a **schedule S** — a list of operations (reads, writes) from several transactions, interleaved together.

Example:

```

r1(X)
w1(X)
r2(X)
w2(X)
r1(Y)
w1(Y)

```

Our goal:

👉 To check whether this interleaved schedule behaves *as if* transactions had executed one after another — i.e., serially.

If yes → ✅ It's **conflict-serializable**.

🧩 2 The key idea: Precedence graph (Serialization graph)

We'll represent dependencies between transactions using a **directed graph**:

- Each **transaction T_i** is a **node**.
- A **directed edge ($T_i \rightarrow T_j$)** means " T_i must come before T_j " in any equivalent serial order.

Why? Because in the schedule, T_i did something that *conflicts* with T_j on the same data item.

🧩 3 When to draw an edge?

According to **Algorithm 21.1**, an edge ($T_i \rightarrow T_j$) is drawn whenever T_i 's operation happens **before** a conflicting operation of T_j on the same item.

Conflicts happen in 3 cases:

Case	Operations	Meaning	Edge Direction
1	<code>write_item(X)</code> in $T_i \rightarrow$ <code>read_item(X)</code> in T_j	T_i wrote, then T_j read same X	$(T_i \rightarrow T_j)$
2	<code>read_item(X)</code> in $T_i \rightarrow$ <code>write_item(X)</code> in T_j	T_i read, then T_j wrote same X	$(T_i \rightarrow T_j)$
3	<code>write_item(X)</code> in $T_i \rightarrow$ <code>write_item(X)</code> in T_j	T_i wrote, then T_j wrote same X	$(T_i \rightarrow T_j)$

4 Example: Schedule D from earlier

Schedule D:

```
T1: r1(X), w1(X), r1(Y), w1(Y)
T2: r2(X), w2(X)
```

Let's check conflicts step-by-step:

Operation	Conflict Type	Edge
w1(X) before r2(X)	Write-Read	T1 → T2
w1(X) before w2(X)	Write-Write	T1 → T2

No other conflicts (Y only touched by T1).

✓ Only one direction: **T1 → T2**

So, our **precedence graph** looks like this:

```
T1 → T2
```

5 Check for cycles

Now the rule:

| If the precedence graph has no cycle, the schedule is conflict-serializable.

Here, no cycles → so it's serializable.

Equivalent serial schedule = **T1 followed by T2**.

6 What if there were cycles?

Suppose we had:

```
T1: r1(X), w1(X)
T2: r2(Y), w2(Y)
T3: r3(X), w3(Y)
```

If operations interleave like this:

```
T1: w1(X)
T3: r3(X)
T3: w3(Y)
T2: r2(Y)
```

We'd get edges:

- $w1(X) \rightarrow r3(X) \Rightarrow T1 \rightarrow T3$
- $w3(Y) \rightarrow r2(Y) \Rightarrow T3 \rightarrow T2$
- (and if T2 wrote something before T1's read, maybe $T2 \rightarrow T1$)

If we ever end up with a loop like:

```
T1 → T3 → T2 → T1
```

then ✗ not serializable.

Because you can't find any serial order that satisfies *all* those dependencies.

7 What does this algorithm give us?

Step	Result
Build graph	Shows dependency between transactions
Check for cycles	Determines serializability
Find topological order	Gives equivalent serial schedule

▼ How Serializability Is Used for Concurrency Control

What's the main point here?

So far, we learned:

- A **serializable schedule** = correct schedule (same effect as some serial order)
- But actually **testing** serializability (using precedence graphs, etc.) **after execution** is **impractical** in real systems.

Now we'll see:

- Why testing is hard
 - What DBMSs do instead (use protocols)
-

1 Why is testing serializability directly not practical?

Let's imagine the system is running many transactions at once.

- OS and DBMS interleave their read/write operations dynamically (based on CPU scheduling, I/O waits, priorities, etc.).
- The **order** of interleaving isn't fixed — it depends on timing and load.
- You **can't know in advance** what the final schedule will look like.

So, even if we *could* test for serializability using the precedence graph method (Algorithm 21.1), that would only work **after** the schedule is formed.

But by then:

- Transactions may have already modified data.
- If we find that the schedule is **not serializable**, we'd have to **undo everything** — which is obviously **inefficient and unsafe**.

👉 Hence, testing serializability *after the fact* is **not feasible**.

2 What do DBMSs do instead?

Instead of testing, **DBMSs enforce rules (protocols)** that **guarantee** serializable schedules automatically.

These are called **concurrency control protocols**.

4 The role of the *committed projection* $C(S)$

They mention this again here:

"The committed projection $C(S)$ of a schedule S includes only the operations in S that belong to committed transactions."

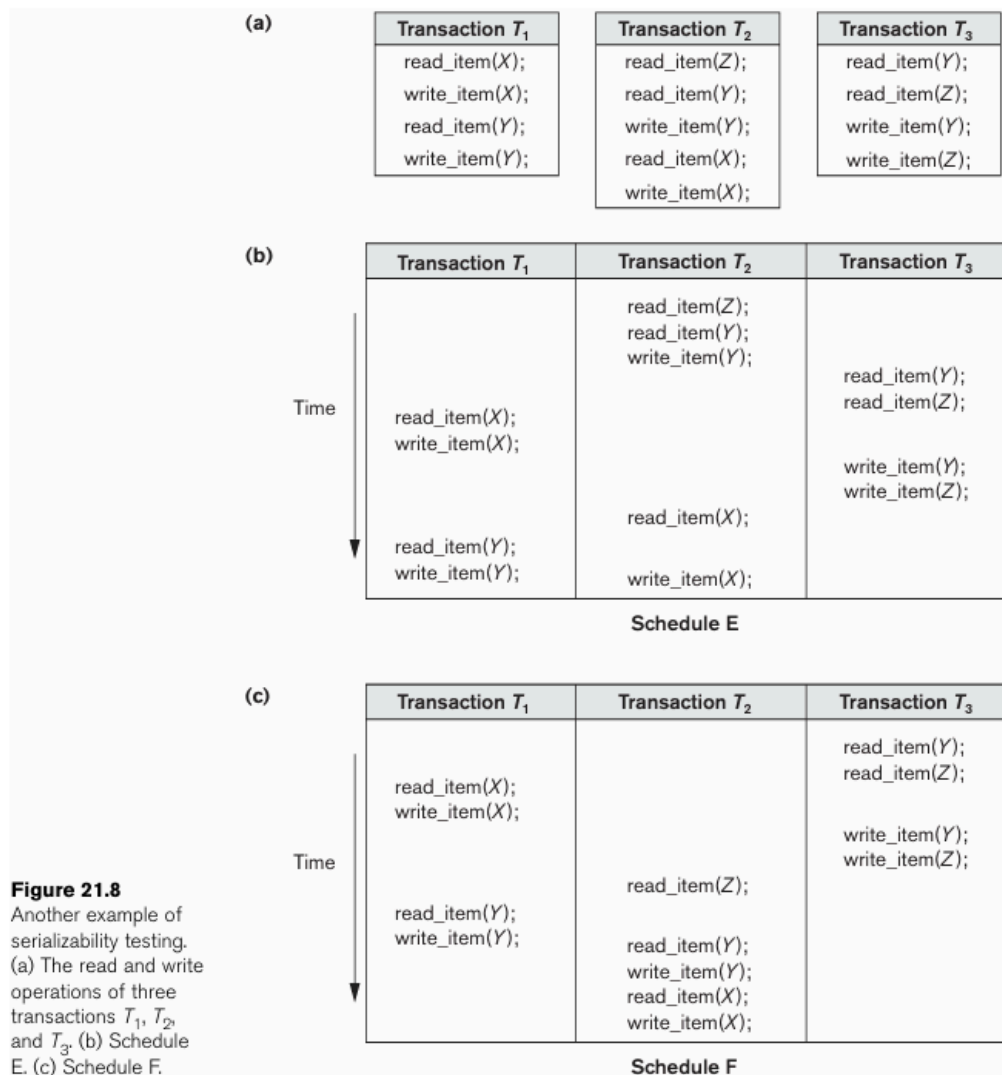
Why?

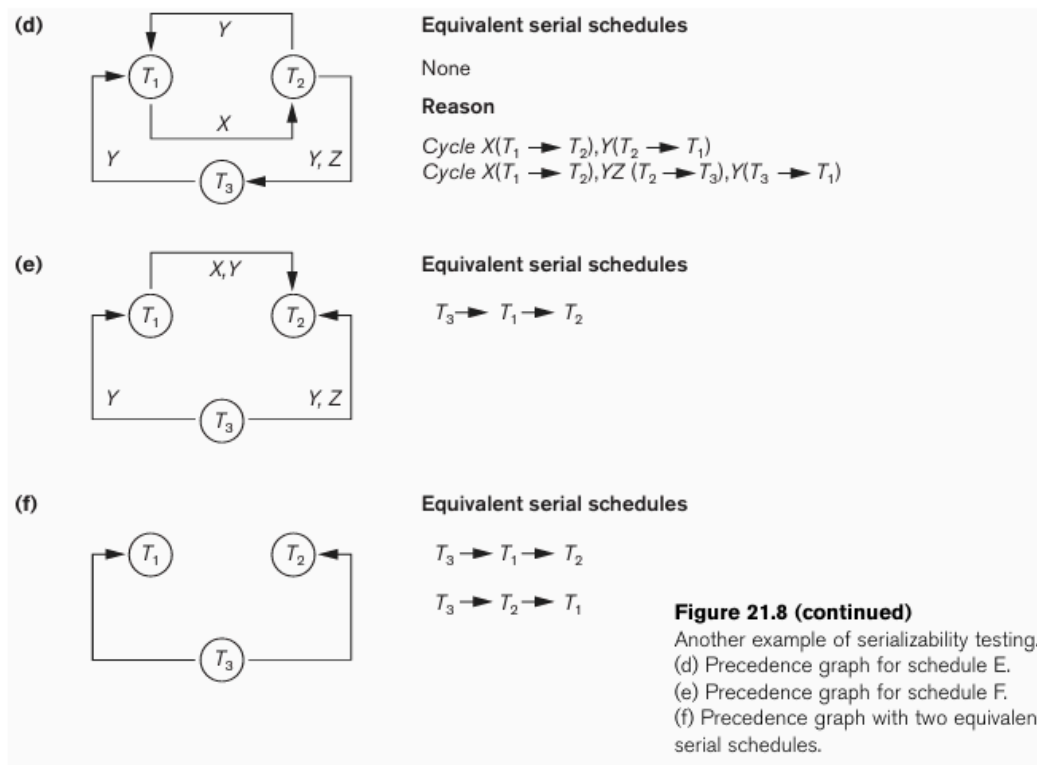
Because in reality:

- Many transactions may be **aborted** or **still running**.
- The DBMS only guarantees **consistency** for **committed transactions**.
- So, we only need to check serializability of the **committed portion** of the schedule.

👉 In practice, we define a schedule S to be **serializable** if its **committed projection C(S)** is equivalent to a serial schedule.

Figure 21.8 Introduction to transaction recovery: concepts and theory





▼ View Equivalence and View Serializability

We already know about **conflict serializability** (checking conflicts between read/write operations).

Now, there's another way to define if two schedules are "equivalent":

👉 **View equivalence** — it's *less strict* than conflict equivalence.

It leads to a new concept: **view serializability**.

2 Definition of View Equivalence

Two schedules **S** and **S'** are *view equivalent* if these **three conditions** hold:

(1) Same transactions and operations

Both schedules involve the **same transactions** doing the **same operations** — only the *order* can differ.

(2) Reads see the same writes

If a transaction **T_i** reads some item **X** from another transaction's write **w_j(X)** (or the original value of X),

then the same must be true in both schedules.

👉 Each read must “see” the same value in both schedules.

(3) Same final writes

If a transaction T_k is the one that does the **last write** on an item Y in schedule S , then it must also be the last to write Y in schedule S' .

👉 This ensures the *final database state* is the same.

3 Intuitive Meaning

If every read operation in both schedules reads the *same value* (from the same write),

and the *final write* on every item is by the same transaction,

then both schedules “look the same” to all transactions —

they see the same **view** of the data.

Hence, the two schedules are **view equivalent**.

A schedule is called **view serializable** if it is view equivalent to some **serial schedule** (one where transactions happen one after another).

4 Relation between View and Conflict Serializability

If there’s a rule called the **constrained write assumption** (or **no blind writes**) — meaning every write is *based on a prior read* of that same item —

then **view serializability** and **conflict serializability** are the same.

Example:

If a transaction writes x , it must have read x first and changed it (like $x = f(x)$).

But if **blind writes** are allowed (writing without reading),

then view serializability can allow more schedules than conflict serializability.

5 Example (Schedule Sg)

Given:

```
T1: r1(X); w1(X);  
T2: w2(X);
```

```
T3: w3(X);  
Schedule Sg: r1(X); w2(X); w1(X); w3(X); c1; c2; c3;
```

Here,

`w2(X)` and `w3(X)` are **blind writes** — they write `X` without reading it first.

👉 This schedule is **view serializable** (equivalent to serial $T1 \rightarrow T2 \rightarrow T3$)

but **not conflict serializable** (since their writes overlap and conflict order can't be rearranged safely).

So, **every conflict-serializable schedule is also view-serializable**,

but **not all view-serializable schedules are conflict-serializable**.

6 Testing View Serializability

You can test if a schedule is view serializable,

but that test is **NP-hard**, meaning it's *computationally very expensive* (no fast algorithm exists).

✓ In summary:

Concept	Based on	Restrictiveness	Example
Conflict Serializability	Conflicting read/write order	More strict	Easier to check
View Serializability	What each transaction "sees" (same reads & final writes)	Less strict	Harder to check, allows blind writes

▼ Other Types of Equivalence

1 Main Idea — Other Types of Equivalence

So far, we studied **serializability** (conflict or view) as the condition that makes a schedule "correct."

But now, the book says that **serializability can sometimes be too strict** — meaning it unnecessarily blocks schedules that are still logically correct.

There are certain applications where **non-serializable schedules** can still give **correct results**.

2 Example — Debit–Credit Transactions

Example: **bank deposits and withdrawals.**

A transaction like:

- Deposit → adds money to an account
- Withdraw → subtracts money from an account

Here, operations are of the form:

balance = balance + X

or

balance = balance - X

These are **commutative operations**, meaning:

👉 You can add or subtract in *any order* and the final result will be the same.

Example:

```
(balance + 10) - 20 = (balance - 20) + 10
```

So even if the transactions run in overlapping order, the outcome is still correct.

3 Given Transactions

We have:

```
T1: r1(X); X := X - 10; w1(X); r1(Y); Y := Y + 10; w1(Y);  
T2: r2(Y); Y := Y - 20; w2(Y); r2(X); X := X + 20; w2(X);
```

T1 transfers ₹10 from X → Y.

T2 transfers ₹20 from Y → X.

4 Non-serializable Schedule (Sh)

The schedule:

```
Sh: r1(X); w1(X); r2(Y); w2(Y); r1(Y); w1(Y); r2(X); w2(X);
```

This is **not serializable**, because the operations of T1 and T2 are interleaved in such a way that you can't reorder them into a clean serial form without conflicts.

But, because each transaction's operations on a given item (`r`, update, `w`) are **grouped together** and **commutative**,

the final result will still be correct — balances will reflect all deposits and withdrawals properly.

5 Why This Works

The **key condition**:

For each data item (like X or Y), the group of operations

(`read`, `update`, `write`) from one transaction **must not be interrupted** by another transaction's conflicting operation on the same item.

As long as that happens,

→ order of applying "+amount" and "-amount" doesn't matter (because of commutativity).

So even though `Sh` is **not serializable**,

it's still **correct** given the meaning (semantics) of the operations.

6 Practical Implications

Researchers realized that **strict serializability** sometimes blocks these harmless interleavings.

So they began working on **weaker (more relaxed)** forms of concurrency control that still ensure correctness *without requiring full serializability*.

7 Real-world Example — Long Transactions

In some areas (like **CAD — Computer-Aided Design** for airplanes, etc.), transactions can take a **very long time** — hours or days.

In those cases, strict serializability is **too restrictive** and **inefficient**.

So **relaxed concurrency control schemes** are used,

allowing some interleaving while keeping the database consistent.

Concept	Meaning	Example
Serializability	Transactions appear to run one after another	Banking, normal DBMS
Too restrictive	Disallows some safe interleavings	
Relaxed correctness	Allows safe interleaving using semantics (like addition/subtraction)	Debit-credit operations
Result	Schedule may not be serializable, but still correct	Sh schedule